

演算法報告

楊文德

✚ 程式碼設計(依照出發順序)

```
void observe_RM_quicksort(vector<int> &v, int left, int right)
{
    double time = 0;
    LARGE_INTEGER nFreq, nBeginTime, nEndTime;
    QueryPerformanceFrequency(&nFreq); // 取得計時器頻率
    QueryPerformanceCounter(&nBeginTime); // 開始計時

    RM_Quicksort(v, left, right); // 執行合併排序

    QueryPerformanceCounter(&nEndTime); // 停止計時
    time = (double)(nEndTime.QuadPart - nBeginTime.QuadPart) / (double)nFreq.QuadPart;
    cout << fixed << setprecision(6) << "RM Quick Sort Time : " << time << " seconds"
}
```

首先我先設定一個 `observe_RM_quicksort` 函數方便觀察 `RM_quicksort`，我是使用 `<windows.h>` 中的 `QueryPerformance` 在微秒的精度下測量 `RM_quicksort` 的運算時間。

```
void RM_Quicksort(vector<int> &v, int left, int right)
{
    if (left < right)
    {
        int pivot = Randomized_Partition(v, left, right);
        RM_Quicksort(v, left, pivot - 1);
        RM_Quicksort(v, pivot + 1, right);
    }
}
```

在 `RM_quicksort` 中我用 `divide` 的技巧，將一整個 `array` 以 `pivot` 為界線切分成左右 `array`，然後不一樣的是 `pivot` 現在變成用 `random` 的方式選擇。

```
int Randomized_Partition(vector<int> &v, int left, int right)
{
    int i = left + rand() % (right - left + 1);
    swap(v[right], v[i]);
    return Partition(v, left, right);
}
```

雖然原則上 `pivot` 還是以最右邊的元素為主，但為了讓 `quicksort` 變得隨機化，我們用 `rand()` 亂數種子隨機產生 `[left, right]` 之間的數字，最後再與最右邊元素交換，這樣就可以保證 `pivot` 的選擇是 `random`。

```

int Partition(vector<int> &v, int left, int right)
{
    int tmp = v[right];
    int i = left - 1;
    for (int j = left; j <= right - 1; j++)
    {
        if (v[j] <= tmp)
        {
            i++;
            swap(v[j], v[i]);
        }
    }
    swap(v[i + 1], v[right]);

    return i + 1;
}

```

最後用原 quicksort 邏輯排法即可。

✚ 運算時間比較

```

Please enter the size of an array : 100
Heap Sort Time : 0.000016 seconds
Insertion Sort Time : 0.000026 seconds
Merge Sort Time : 0.000135 seconds
RM Quick Sort Time : 0.000013 seconds

```

```

Please enter the size of an array : 1000
Heap Sort Time : 0.000361 seconds
Insertion Sort Time : 0.002296 seconds
Merge Sort Time : 0.001551 seconds
RM Quick Sort Time : 0.000228 seconds

```

```

Please enter the size of an array : 10000
Heap Sort Time : 0.004860 seconds
Insertion Sort Time : 0.171231 seconds
Merge Sort Time : 0.016407 seconds
RM Quick Sort Time : 0.001895 seconds

```

```

Please enter the size of an array : 50000
Heap Sort Time : 0.021968 seconds
Insertion Sort Time : 3.815704 seconds
Merge Sort Time : 0.064099 seconds
RM Quick Sort Time : 0.012665 seconds

```

```
Please enter the size of an array : 100000
Heap Sort Time : 0.051474 seconds
Insertion Sort Time : 15.341700 seconds
Merge Sort Time : 0.192784 seconds
RM Quick Sort Time : 0.028213 seconds
```

```
Please enter the size of an array : 200000
Heap Sort Time : 0.118624 seconds
Insertion Sort Time : 89.868898 seconds
Merge Sort Time : 0.268534 seconds
RM Quick Sort Time : 0.068102 seconds
```

比較

從我的測資運算時間終我們可以觀察到 random-quicksort 相較於其他排序法有著更快的速度，理論上與 merge sort、heap sort 有著相同的時間複雜度 $O(n \log n)$ ，但相較於 merge sort 有著更少的空間複雜度，而且因為 random-quicksort 有加入亂數的關係，因此相較於其他 sort 也更來的有效率(期望值可推知)。

時間複雜度分析

理論上:

$$f(n) \leq \begin{cases} O(1) & \text{if } n \leq 1, \\ \Theta(n) + \frac{1}{n} \sum_{p=1}^n (f(p-1) + f(n-p)) & \text{otherwise.} \end{cases}$$

假設對於所有的 $n' < n$ ，都有 $f(n') \leq cn' \log n'$ 。我們想證明 $f(n) \leq cn \log n$ 。而證明的關鍵，是利用後面總和的部分想辦法榨出一個 $-c_0 n$ ，消去前面的項。

$$\begin{aligned} f(n) &\leq c_0 n + \frac{2}{n} \sum_{i=1}^{n-1} f(i) \\ &= c_0 n + \frac{2}{n} \left(\sum_{i=1}^{\lfloor n/2 \rfloor} f(i) + \sum_{i=\lfloor n/2 \rfloor+1}^{n-1} f(i) \right) \\ &\leq c_0 n + \frac{2}{n} \left(\sum_{i=1}^{\lfloor n/2 \rfloor} ci \log \frac{n}{2} + \sum_{i=\lfloor n/2 \rfloor+1}^{n-1} ci \log n \right) \\ &= c_0 n + \frac{2}{n} \left(\sum_{i=1}^{\lfloor n/2 \rfloor} ci (\log n - 1) + \sum_{i=\lfloor n/2 \rfloor+1}^{n-1} ci \log n \right) \\ &= c_0 n + \frac{2}{n} \left(\sum_{i=1}^{n-1} ci \log n - \sum_{i=1}^{\lfloor n/2 \rfloor} ci \right) \\ &= c_0 n + c(n-1) \log n - c \frac{2}{n} \frac{\lfloor n/2 \rfloor (\lfloor n/2 \rfloor + 1)}{2} \\ &\leq c_0 n + cn \log n - \frac{c}{4} n + \left(\frac{c}{4} - c \log n \right) \\ &\leq c_0 n + cn \log n - \frac{c}{4} n \\ &= cn \log n + \left(c_0 - \frac{c}{4} \right) n \end{aligned}$$

如果我們選取 $c \geq 4c_0$ ，那麼便能使式子成立，從而命題得證。

實際上:

透過運算時間上觀察，我們可以發現時間增長不像 `insertion sort` 那般的劇烈，大致上可以看出資料每增長 10 倍，所需的時間也會跟著增長 100 倍 $O(n^2)$ ，很明顯 `RM_quicksort` 運算時間上更近 `Heap`、`merge` 的 $O(n \log n)$ 。

心得

在這次的演算法作業中，加入了以往演算法不曾有的元素—亂數，這相較於其他演算法有著更強的性能、更快的速度，這現象讓我感到非常好奇，雖然用期望值推導結果的確如此，但感覺還不夠直覺。

引用

時間複雜度理論推導: <https://tmt514.github.io/algorithm-analysis/sorting/randomized-quicksort.html>